

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Nasarawa State, Nigeria

LAB REPORT

Lab 01: Introduction to Programmable Logic Controllers

Item	Details
Lab Title	Lab 01: Introduction to Programmable Logic Controllers
Student Name	Sunusi Abdullahi
Student ID (Reg No)	C11/26/ACIS/17362
Course Name	Introduction to Programmable Logic Controllers
Date of Submission	30/09/2026
Version	Version 1.0

1. Executive Summary

This laboratory introduced the Programmable Logic Controller (PLC). A PLC is a small industrial computer that controls machines such as motors, pumps and valves in factories and in critical infrastructure. All work was done inside the isolated ICDFA training virtual machine.

Four main activities were carried out: confirming that the laboratory environment was working, identifying the main parts of a PLC, classifying industrial input and output devices, and opening a motor-control program in the OpenPLC Editor.

The main outcomes were these. The major PLC components and their jobs were identified. Devices were sorted into inputs and outputs, and discrete signals were separated from analogue signals. The Motor_Control ladder diagram was inspected: pressing Start switches the motor on, a latching contact keeps it running after Start is released, and pressing Stop switches it off. The report also explains why connecting PLCs to networks is useful but brings cybersecurity risks.

2. Lab Objectives

By the end of this lab, I should be able to:

- Explain what a PLC is and how it works.
- Name the main parts of a PLC: processor, memory, power supply, input module, output module, rack/chassis and programming device.
- Tell the difference between discrete (digital) and analogue I/O signals.
- Identify common industrial input and output devices.
- Open OpenPLC Editor and inspect an existing Ladder Diagram project.
- Read and explain the Ladder Logic used for motor control.
- Explain the cybersecurity effects of connecting PLCs to networks.

3. Tools and Resources Used

Tool / Resource	What it was used for
ICDFA PLC Virtual Machine (IP 192.168.1.10)	The training PLC environment supplied by ICDFA. All practical work was done inside it.
Virtualisation platform: (version)	Used to import and run the PLC VM on a Host-Only (isolated) network. Write the name and version you used.
OpenPLC Editor (version	The program used to open the Ladder Diagram project and view the Motor_Control logic.
Terminal (in the VM)	Used to check the IP address of the VM.
Screenshot tool	Used to save evidence at each important step.

4. Methodology (Step-by-Step Procedure)

I followed the steps in the lab brief in order. A screenshot was captured at each important step. The screenshots are numbered as Figures 1 to 8.

4.2 The Steps

Step 1: Import the VM and set the network. I imported the IC DFA PLC VM into the virtualisation platform. I set the network adapter to Internal / Host-Only. I did not use Bridged, so the lab stays isolated from real networks.

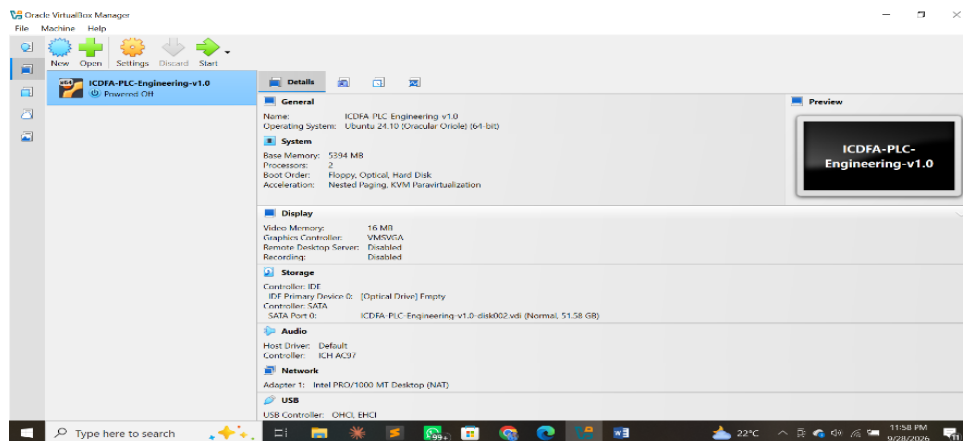


Figure 1: VM network adapter set to Host-Only (isolated).

Step 2: Create a snapshot and start the VM. Before making any change, I created a snapshot named **LAB01-START**. Then I started the PLC VM and signed in with the laboratory account given to me.

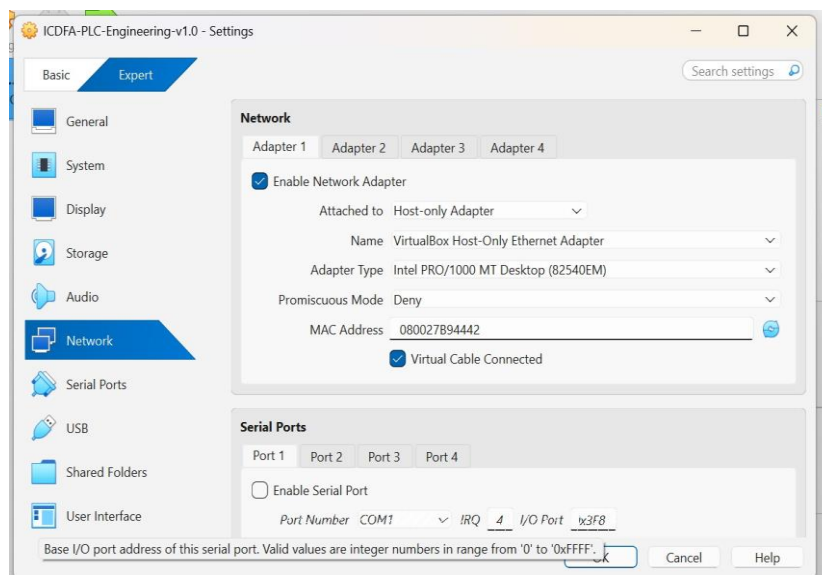
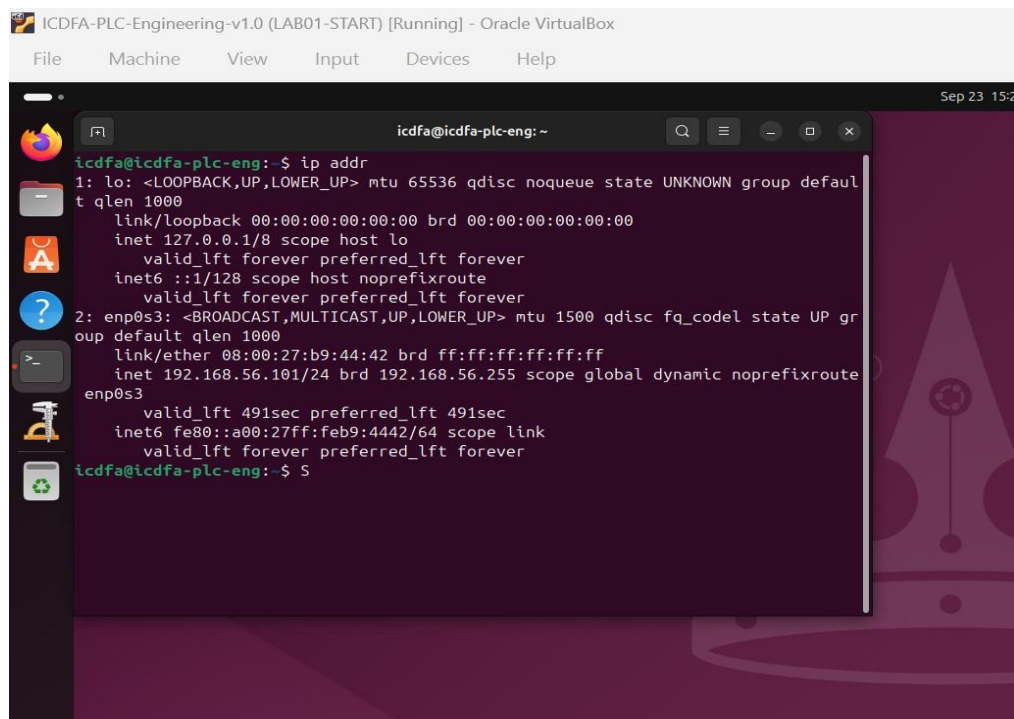


Figure 2: Clean snapshot LAB01-START created before starting.



```
icdfa@icdfa-plc-eng: ~  
icdfa@icdfa-plc-eng:~$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:b9:44:42 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3  
        valid_lft 491sec preferred_lft 491sec  
    inet6 fe80::a00:27ff:feb9:4442/64 scope link  
        valid_lft forever preferred_lft forever  
icdfa@icdfa-plc-eng:~$ S
```

Figure 3: ICDFA PLC virtual machine running and signed in.

Step 3: Confirm the IP address. I opened a Terminal and typed the command below to see the address of the VM. It should show 192.168.1.10. If my VM shows a different address, I will write that address here and will not change it:

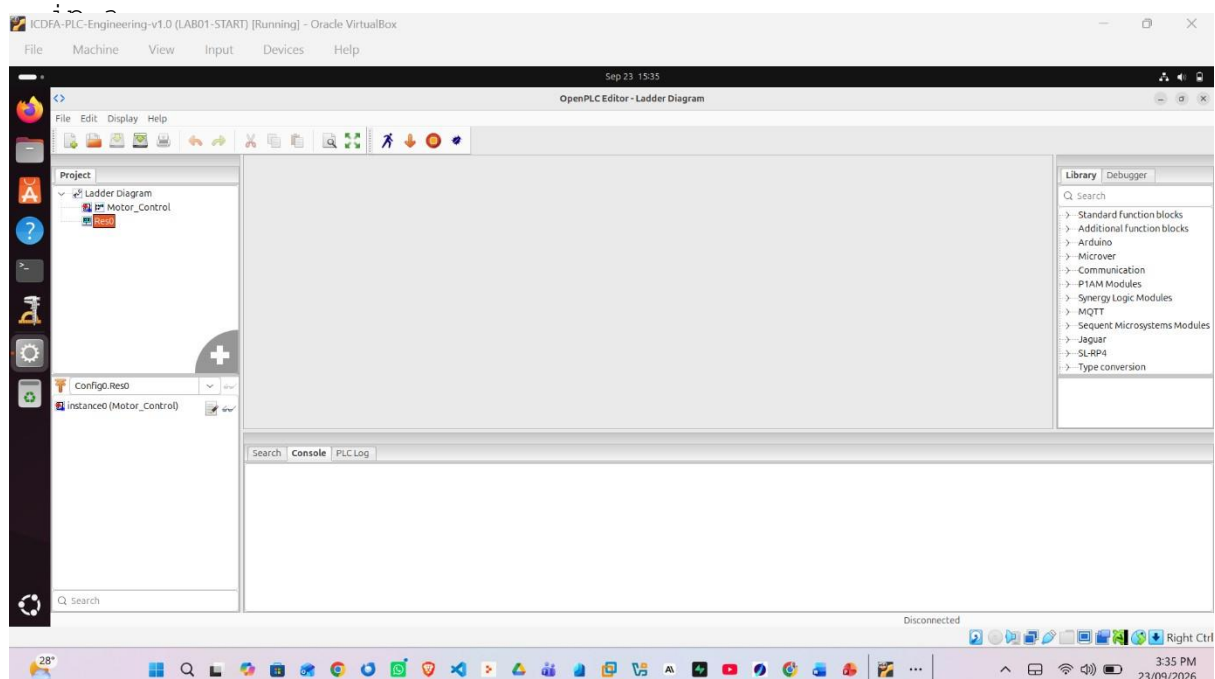


Figure 4: VM IP address confirmed as 192.168.1.10.

Step 4: Open OpenPLC Editor. I opened OpenPLC Editor from the desktop / application menu.

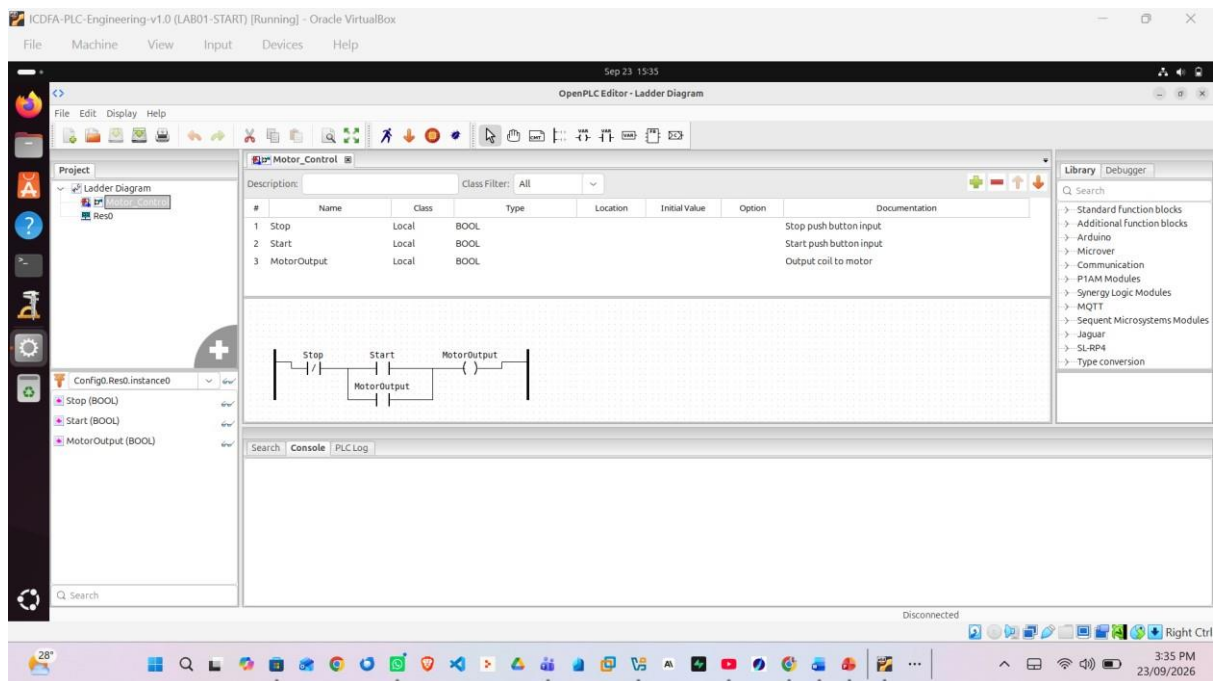


Figure 5: OpenPLC Editor launched.

Step 5: Open the Ladder Diagram project. I clicked **File > Open**. I browsed to the **PLC Programming Language Examples** folder and opened the **Ladder Diagram** project.

Step 7 and Step 8: Identify and trace the logic. I identified the rails, the rung, the Start contact, the Stop contact, the MotorOutput contact and the MotorOutput coil. Then I traced the logic from the left rail to the right rail. The full working is in Evidence 4 (Section 5.4).

Step 9: Record I/O examples. I wrote one example each of a discrete input, analogue input, discrete output and analogue output. The full working is in Evidence 3 (Section 5.3).

Step 10: Finish. I saved my evidence and this report. Then I restored the VM to the LAB01-START snapshot / powered it off as directed by the instructor.

5. Evidence and Analysis

5.1 Evidence 1: Laboratory Environment and PLC Identification

Question: Show that the IC DFA PLC virtual machine / OpenPLC environment was accessed.

Answer: The screenshots below show the working laboratory environment.

- Figure 3 shows the IC DFA PLC virtual machine running and signed in.
- Figure 4 shows the VM address 192.168.1.10, which is the expected training address for the PLC VM.
- Figure 5 shows OpenPLC Editor opened inside the same VM.

So the required laboratory environment was accessed, and it was kept on an isolated Host-Only network (Figure 1).

5.2 Evidence 2: PLC Architecture Analysis

Question: Identify the major components of a PLC and explain the job of each one.

Simple way to remember: Think of the PLC as a human being. The CPU is the brain, memory is the notebook, the power supply is the food that gives energy, the input module is the eyes and ears, the output module is the hands and legs, and the rack is the body that holds everything together.

Component	What it does (simple words)	Human example
1. Processor / CPU	Reads the input signals, runs the stored program, and decides which outputs must be ON or OFF. It repeats this again and again very fast (scan cycle).	The brain
2. Memory	Stores the user program (the ladder logic) and the current data, such as the state of inputs and outputs.	The notebook
3. Power supply	Changes the mains supply to the low DC voltage needed by the CPU and the modules.	Food / energy
4. Input module	Receives signals from field devices (push buttons, sensors, switches) and changes them into signals the CPU can understand.	Eyes and ears
5. Output module	Takes the CPU decision and sends power or signal to the load devices (motor contactor, lamp, valve).	Hands and legs
6. Rack / chassis	The frame that holds the CPU, power supply and I/O modules in place and connects them together.	The body / skeleton
7. Programming interface / device	The computer and software (here, OpenPLC Editor) used to write, load and view the PLC program.	The blackboard where the instruction is written
Expansion modules (extra)	Additional I/O modules added when more inputs or outputs are needed than the main unit has.	Extra hands

How the parts work together (order): Input device gives signal > Input module > CPU (uses program in memory) > Output module > Output device works. The power supply feeds the rack and every module.

5.3 Evidence 3: Input / Output Classification

Question: Give at least three PLC input devices and three output devices, classify each as input or output, and explain discrete versus analogue I/O.

Part A: Classification table

Rule I used: If the device sends information INTO the PLC, it is an INPUT. If the PLC sends a command OUT to the device, it is an OUTPUT (load).

No.	Device	Input or Output	Signal type	Example use in industry
1	Push button (Start)	Input	Discrete	Operator presses to start a motor.
2	Limit switch	Input	Discrete	Shows that a door or conveyor gate is at its end position.
3	Proximity sensor	Input	Discrete	Detects that an object has reached a point on the conveyor.
4	Temperature transmitter (4 to 20 mA)	Input	Analogue	Sends the temperature of a tank to the PLC.
5	Motor contactor / starter	Output	Discrete	Switches the motor ON or OFF.
6	Indicator lamp	Output	Discrete	Shows that the motor is running.
7	Solenoid valve	Output	Discrete	Opens or closes a water or air line.
8	Variable speed drive speed reference (4 to 20 mA)	Output	Analogue	Sets how fast a motor or pump should run.

Step 9 answer (one example each):

Type	Example from an industrial process
Discrete input	Push button (Start / Stop) or limit switch
Analogue input	Temperature transmitter (4 to 20 mA)
Discrete output	Motor contactor or indicator lamp
Analogue output	Speed reference sent to a variable speed drive (4 to 20 mA)

Part B: Discrete I/O versus Analogue I/O

Point	Discrete (Digital) I/O	Analogue I/O
Meaning	The signal has only TWO states.	The signal can have MANY values in a range.
Values	ON or OFF (1 or 0).	Continuous, for example 4 to 20 mA or 0 to 10 V.
Example	Push button pressed or not pressed.	Temperature, pressure, level, speed.
Simple picture	Like a light switch.	Like a dimmer or a thermometer.

Worked example (analogue signal): A temperature transmitter gives 4 mA at 0 degrees C and 20 mA at 100 degrees C. What temperature is shown when the signal is 12 mA?

Step 1: Range of the signal = $20 - 4 = 16$ mA.

Step 2: The signal above the start = $12 - 4 = 8$ mA.

Step 3: Fraction of the range = $8 / 16 = 0.5$.

Step 4: Temperature = $0.5 \times 100 = 50$ degrees C.

Answer: 12 mA means 50 degrees C. A discrete input could never show this, because it only knows ON or OFF.

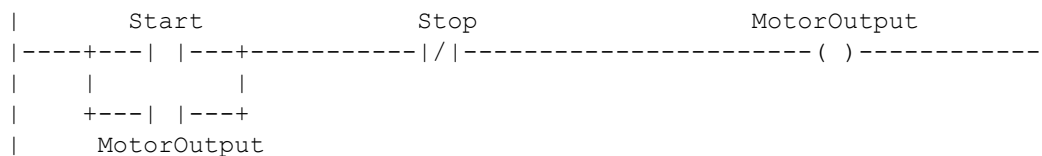
5.4 Evidence 4: OpenPLC and Ladder Logic Practical

Question: Open the Motor_Control program, show the ladder logic (Figure 8), and explain in my own words how it controls the output.

Part A: What I identified in the program

First, the ladder diagram is like an electric circuit turned on its side. Power flows from the left rail to the right rail. If a complete path is found, the coil at the end is energised.

Blackboard sketch of the logic (this is my simple drawing; the OpenPLC screen in Figure 8 is the real one):



Key: **| |** = normally open (NO) contact, **|/|** = normally closed (NC) contact, **(/)** = output coil.

Check on your screen: the names and positions may look slightly different in OpenPLC. Confirm that Stop is drawn with a slash (normally closed), and that the MotorOutput contact is drawn in parallel (under) with Start. If Stop is placed in a slightly different place on the same line, the logic is still the same.

Element	Symbol	Its job
Left and right power rails	Two vertical lines	The left rail is the "power in" and the right rail is the "return". The logic is traced from left to right.
Rung	Horizontal line between the rails	One complete line of logic. This program has one motor-control rung.
Start contact	NO contact 	Normally open. It closes only while the Start button is pressed. It gives the starting signal.
Stop contact	NC contact / 	Normally closed. It conducts all the time. When the Stop button is pressed it opens and cuts the path.
MotorOutput contact (latch)	NO contact , parallel with Start	It copies the state of the coil. When the motor is ON it closes, and so keeps the path complete after Start is released. This is called latching or seal-in.
MotorOutput coil	Coil (/)	The output. When power reaches it, the coil is ON and the motor is ON.

Part B: Tracing the logic step by step

I trace the path from the left rail to the right rail for each situation.

Case 1: Nothing is pressed (starting condition). Start is open. The MotorOutput contact is open because the motor is OFF. So there is no path to the coil. **Motor is OFF.**

Case 2: Press Start. Start closes. The Stop contact is normally closed, so it lets power pass. A complete path is made: left rail > Start > Stop > coil > right rail. The coil is energised. **Motor is ON.** At the same moment, the MotorOutput contact closes.

Case 3: Release Start. Start opens again. But the MotorOutput contact is now closed, so power still passes through the parallel branch: left rail > MotorOutput contact > Stop > coil > right rail. **Motor stays ON.** This is the latch.

Case 4: Press Stop. The normally closed Stop contact opens. The path is broken for both branches, so the coil loses power. **Motor is OFF.** The MotorOutput contact opens too.

Case 5: Release Stop. Stop closes again, but Start is open and the MotorOutput contact is open. There is still no path. **Motor stays OFF** until Start is pressed again.

Summary table:

Case	Start	Stop	MotorOutput contact (before)	Coil	Motor
1	Not pressed	Not pressed	Open	OFF	OFF
2	Pressed	Not pressed	Open	ON	ON
3	Released	Not pressed	Closed	ON	ON (latched)
4	Any	Pressed	Any	OFF	OFF
5	Released	Released	Open	OFF	OFF

Part C: My conclusion in my own words

What must be true for the motor to switch ON: the Start button must be pressed (or the latch must already be closed) AND the Stop button must not be pressed.

What makes the motor switch OFF: pressing the Stop button. This opens the normally closed Stop contact and breaks the path to the coil.

5.5 Evidence 5: Reflection and Knowledge Check

Part A: Reflection (network-connected PLCs)

Connecting PLCs to a network gives clear operational benefits. Engineers can monitor machines from a control room instead of walking to each one, so problems are noticed faster. Programs can be updated remotely, data can be recorded for reports, and many machines can work together as one system. This saves time and improves production.

However, the same connection creates cybersecurity considerations. A PLC was built to control machines, not to defend itself, and many older PLCs have no password or a very weak one. If a PLC can be reached from the office network or the internet, an attacker may change its logic, stop a motor, or send false readings. The effect is not only stolen data. It can be stopped production, damaged equipment, or danger to people. In critical infrastructure such as water supply or power stations, a whole community can be affected.

For this reason, PLC networks should be kept separate from office and public networks, exactly as this lab was done on an isolated Host-Only network. Other good practices are

strong passwords, limited user access, regular updates and monitoring of network traffic. Convenience must never come before safety.

Part B: Knowledge Check

1. What is the difference between an input module and an output module?

An input module brings signals from field devices (push buttons, sensors, switches) into the PLC and changes them into a form the CPU can read. An output module does the opposite: it takes the decision of the CPU and sends power or a signal out to load devices such as motors, lamps and valves.

2. Why is a normally closed contact commonly associated with stop or emergency-stop logic?

A normally closed contact conducts all the time and opens when the Stop button is pressed. So pressing Stop breaks the path and switches the motor off. It is also safer (fail-safe): if the Stop wire breaks or is disconnected, the circuit opens and the motor stops. With a normally open stop contact, a broken wire would go unnoticed and the motor could not be stopped.

3. Give one example of a discrete I/O signal and one example of an analogue I/O signal.

Discrete: a push button (pressed = ON, not pressed = OFF). Analogue: a temperature transmitter sending 4 to 20 mA.

4. What operational risk can arise when industrial automation devices become network-connected?

An unauthorised person or malware could reach the PLC through the network and change its program or commands. This can stop production, damage equipment, or cause unsafe conditions. Network faults or delays can also affect control and monitoring.

6. Conclusion

This lab showed that a PLC reads inputs, runs its stored program, and controls outputs. The main parts are the CPU, memory, power supply, input and output modules, and the rack, and they are programmed through a programming device such as OpenPLC Editor. Discrete signals have two states, while analogue signals have a range of values. The Motor_Control ladder diagram showed how a Start contact, a normally closed Stop contact and a latching contact can start, hold and stop a motor safely. The lab also showed that PLCs used in critical infrastructure must be protected when they are connected to networks.

7. Recommendations

- Keep PLC and training environments on isolated networks. Never bridge them to office or public networks.
- Use normally closed contacts for stop and emergency-stop functions so that a broken wire stops the machine.

- Protect real PLCs with strong passwords, limited access and regular updates, and monitor their network traffic.
- Create a clean snapshot before every lab and save screenshots as the work progresses.

9. References

International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab 01: Introduction to Programmable Logic Controllers (Student Edition). Intro to Critical Infrastructure for IT Learners. <https://www.icdfa.edu.ng>

OpenPLC Project. OpenPLC Editor documentation. <https://openplcproject.com>

National Institute of Standards and Technology (NIST). (2023). Guide to Operational Technology (OT) Security (SP 800-82 Rev. 3).

International Electrotechnical Commission (IEC). IEC 61131-3: Programmable controllers, Part 3: Programming languages.